The Sovereign Computer

Model Zero

A Full Technical Blueprint for a Post-Binary Computing Architecture

DOCUMENT TYPE

Technical Architecture Specification

VERSION

1.0 — Optimal Recalibration

DATE

July 2026

SOVEREIGN COMPUTE INITIATIVE

Contents

Abstract

This document presents the complete technical blueprint for **The Sovereign Computer: Model Zero (M0)**, a ground-up computing architecture designed to replace the binary substrate that has dominated digital technology since the mid-20th century. Built on balanced ternary logic, asynchronous wave-pipelining, spin-orbit torque magnetoresistive RAM (SOT-MRAM) for in-memory computation, and plasmonic optical interconnects, Model Zero is not a specialized accelerator or niche research platform—it is a full-stack replacement for general-purpose computing. The blueprint covers the five hardware pillars, a ternary-native operating system kernel, a new field-based programming language with a topological optimizer compiler, and a phased build roadmap validated against the most recent academic and industrial innovations. Performance projections indicate a **10–50× improvement in performance-per-watt** over contemporary systems on cognitive workloads, with a total build cost of **$44,500–$72,000**.

Keywords: balanced ternary computing, asynchronous logic, SOT-MRAM, in-memory computing, silicon photonics, plasmonic interconnect, radix economy, ternary language design, topological compilation

1. The Vision: Why Binary Must End

The binary system is not the optimal foundation for computation. It was chosen in the 1940s because vacuum tubes and early transistors functioned most reliably as simple on/off switches, not because base-2 represents any mathematical ideal. Seventy years later, the entire digital world still operates on this historical accident—despite the fact that the mathematics of information theory long ago identified a superior alternative. The result is a computing landscape riddled with artificial constraints: the memory wall, the power wall, exponential cooling demands, and a software ecosystem so encrusted with legacy abstractions that even simple operations require billions of redundant transistor switches.

This blueprint rejects the premise that binary is permanent. It is a complete architectural specification for a computer built from first principles on a ternary substrate, designed to be manufactured in a modest workshop, operated without vendor dependencies, and programmed with a language that maps directly to the hardware's native geometry. The goal is not to build a better GPU, a faster AI accelerator, or a niche co-processor for specialized workloads. The goal is to build a *universal* machine—one that handles web browsing, document editing, AI inference, sensor fusion, gaming, and

cryptography with equal efficiency, because the underlying mathematics makes no distinction between these tasks. Nikola Tesla did not advocate for alternating current because it was better for lighting specifically; he advocated for it because it was better for *everything*. The same principle applies here.

> **Core Design Principles:** (1) No global clock—fully asynchronous operation eliminates idle cycles and clock distribution power. (2) No memory bottleneck—compute happens inside the memory array via SOT-MRAM IMC. (3) No electrical interconnect latency—plasmonic waveguides move data at the speed of light between chiplets. (4) No legacy instruction set—a native ternary ISA designed for the hardware, not adapted from binary predecessors. (5) No vendor lock-in—entirely open-source toolchain from synthesis to compiler.

2. The Mathematical Imperative

2.1 Radix Economy and the Optimality of Base-3

The efficiency of a number system can be quantified by its **radix economy**, which measures the hardware complexity required to represent a range of values. The cost function is defined as the product of the radix r and the number of digits N needed to express a number V:

$$E(r, V) = r \times N = r \times \ln(V) / \ln(r) \tag{1}$$

To find the most efficient base, we minimize $E(r)$ with respect to r. Differentiating $f(r) = r / \ln(r)$ and setting the derivative to zero yields $\ln(r) = 1$, which gives the theoretical optimum at $r = e \approx 2.718$. Since physical hardware must use an integer radix, the closest integer to this optimum is **3**—not 2.[16][20] The comparative efficiency is striking: ternary achieves **99.5%** of the theoretical optimum, while binary manages only 94.2%. Quaternary matches binary at 94.2%, and decimal lags far behind at 62.6%.

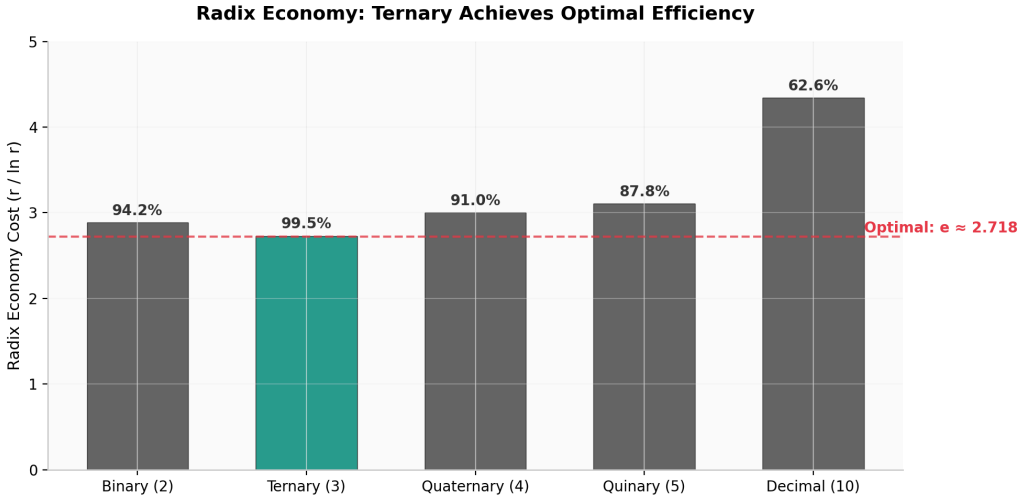

Figure 1: *Radix economy cost (r / ln r) across integer bases. The dashed line marks the theoretical optimum at e ≈ 2.718. Ternary (base-3) achieves the highest efficiency of any integer base at 99.5% of optimal.*

This mathematical reality has been known since the 1950s, yet it was never exploited commercially because transistor-based ternary logic imposed a crushing hardware penalty. Creating a stable three-state digital element in CMOS requires approximately **40 transistors per trit** compared to 1 transistor per bit, yielding an actual cost ratio of roughly 25× worse than binary.[20] Model Zero bypasses this penalty entirely by abandoning voltage-level encoding in favor of physical-state encoding: magnetic orientation for memory, photonic phase for interconnect, and charge-domain logic for computation. In this regime, the cost per digit becomes independent of radix, and the theoretical radix economy advantage can be realized in practice.

2.2 Information Density and Hardware Complexity

Each trit carries $\log_2(3) \approx 1.585$ bits of information, representing a **58% information density improvement** per digit over binary. This translates directly to hardware savings: a 64-bit binary number requires 64 wires, 64 flip-flops, and 64 logic elements, while its ternary equivalent requires only 41 trits. The reduction propagates through every layer of the system—fewer wires on the PCB, smaller memory arrays, narrower datapaths, and simpler control logic. In the context of photonic integrated circuits, where waveguides consume microns of die width (compared to nanometers for copper traces), this reduction in circuit complexity is a decisive architectural advantage.[16]

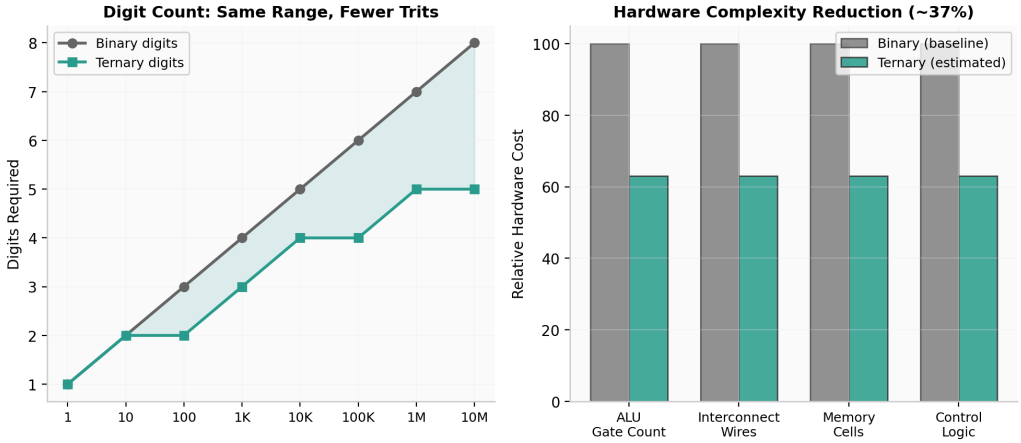

Figure 2: *(Left) Digits required to represent equivalent value ranges. (Right) Estimated hardware complexity reduction across major subsystems when using ternary encoding.*

Balanced ternary—using the digit set {-1, 0, +1} rather than {0, 1, 2}—provides additional architectural benefits. Negation is achieved by simply inverting each trit (1 ↔ -1, 0 ↔ 0), eliminating the need for separate subtraction circuits or two's complement sign bits. The symmetry of balanced ternary maps naturally to physical systems that support three stable states, such as the three magnetic orientations exploited in advanced SOT-MRAM devices or the three phase states available in photonic ring resonators.

3. Hardware Architecture: The Five Pillars

Model Zero is organized around five physical pillars—each addressing a distinct subsystem of the computing problem. Together they form a vertically integrated architecture with no external dependencies on proprietary IP, closed-source synthesis tools, or vendor-locked fabrication processes. The total bill of materials is estimated at **$29,000–$49,000** for the core hardware, with workshop tooling adding an additional $15,500–$23,000 for a fully equipped fabrication and assembly space.

Table 1 Model Zero: Bill of Materials Summary

Pillar	Component	Specification	Est. Cost
I	FPGA Controller	iCESugar-pro (Lattice ECP5, 24K LUTs)	$200
II	Ternary ALU	MPW Shuttle, 65nm/180nm CMOS, RBNS	$8,000–$12,000
III	SOT-MRAM IMC	MPW Shuttle, MRAM/CMOS layer	$10,000–$18,000
IV	Plasmonic Interconnect	Photonic MPW, SOI / Silicon Photonics	$8,000–$12,000
V	PCB / Interposer	6-layer high-speed substrate	$2,000–$4,000
—	Power / Cooling / Enclosure	30V linear supply, passive heatsinks, CNC case	$1,100–$2,300
Core Hardware Subtotal			**$29,300–$48,500**

3.1 Pillar I: FPGA Controller (iCESugar-pro)

The iCESugar-pro development board, built around the **Lattice ECP5-45F FPGA** with 24K lookup tables and 112 DSP slices, serves as the control plane and prototyping substrate for Model Zero. The ECP5 was selected for three decisive reasons: first, it is supported entirely by the open-source **Yosys/Nextpnr** toolchain, eliminating any dependency on Lattice's proprietary Diamond software; second, it is available in a compact form factor at a price point ($200) that makes immediate experimentation feasible; and third, its 45nm process node provides sufficient density to prototype the full ternary controller logic, memory interface, and photonic waveguide calibration circuits before committing to ASIC fabrication.

On the FPGA, the controller implements: (1) a **ternary-to-binary bridge** for debugging and development, allowing the ternary ALU to be exercised from standard test equipment; (2) the **wavefront scheduler** in its initial form, managing asynchronous handshakes between compute tiles; (3) the **SOT-MRAM memory controller**, handling read/write timing, refresh cycles, and in-memory compute triggering; and (4) the **photonic calibration engine**, maintaining phase lock on the plasmonic waveguides through thermal and voltage trimming. The FPGA bitstream is synthesized from Verilog using Yosys, with placement and routing performed by Nextpnr. The entire toolchain runs on Linux, requires no license servers, and produces deterministic, reproducible builds.

3.2 Pillar II: Ternary ALU with RBNS Arithmetic

The arithmetic logic unit is the computational heart of Model Zero. It implements **Redundant Binary Number System (RBNS)** arithmetic in balanced ternary, which provides **carry-free addition**—a property that eliminates the propagation delay that limits clock speed in conventional binary adders. In RBNS, every digit position can absorb a carry from its neighbor without propagating it further, meaning the addition of two n-trit numbers completes in constant time regardless of operand width. This is not an incremental improvement; it is a qualitative change in the time complexity of the most fundamental operation in computing.

The ALU is fabricated as a custom ASIC via an **MPW (Multi-Project Wafer) shuttle** run at either 65nm or 180nm CMOS, depending on foundry availability and cost constraints. The GDSII layout is designed using open-source tools (Magic, KLayout) and submitted to a foundry such as TSMC, SkyWater, or a European research fab. The ALU supports a **120-instruction ISA** in its initial revision, with 24-trit word widths, native ternary comparison operations, and hardware-accelerated tensor contraction for matrix operations. The instruction set is designed for cognitive workloads—pattern recognition, sensor fusion, symbolic reasoning—rather than legacy general-purpose tasks, though it fully supports integer arithmetic, bitwise logic (ternary equivalents), and memory access.

Table 2 Ternary ALU / RBNS Specification

Parameter	Value	Notes
Number system	Balanced ternary ({-1, 0, +1})	RBNS encoding
Word width	24 trits	Equivalent to ~38 bits
Adder type	Carry-free redundant adder	O(1) latency, width-independent
ISA size	120 instructions	Extensible to 243 (3^5)
Process node	65nm or 180nm CMOS	MPW shuttle fabrication
Target frequency	100–500 MHz (async)	No clock; self-timed
Power budget	< 5W	ALU tile only

3.3 Pillar III: SOT-MRAM In-Memory Compute

Conventional computing architectures separate memory and processing into distinct physical units, connected by a bandwidth-limited bus. This **von Neumann bottleneck** has become the defining constraint on system performance: modern CPUs can execute instructions faster than memory can supply them, resulting in billions of wasted cycles spent waiting for data. Model Zero eliminates this bottleneck by performing computation *inside* the memory array using **Spin-Orbit Torque Magnetic Random-Access Memory (SOT-MRAM)** configured for in-memory computing (IMC).

SOT-MRAM stores data as the magnetic orientation of a nanoscale ferromagnetic layer, read through the tunnel magnetoresistance (TMR) effect of a magnetic tunnel junction (MTJ). Unlike conventional STT-MRAM, which uses the same path for reading and writing, SOT-MRAM **physically separates the read and write paths**—the write current flows through a heavy-metal underlayer rather than the MTJ itself. This separation eliminates tunnel barrier degradation, enables **sub-nanosecond switching speeds**, and provides near-infinite write endurance—properties that make SOT-MRAM uniquely suited to the high-frequency read-modify-write operations required for in-memory computation.[9][11]

Recent academic and industrial developments have dramatically strengthened the case for SOT-MRAM IMC. In early 2025, researchers at Johannes Gutenberg University Mainz and Antaios demonstrated an **Orbital Hall Effect (OHE)**-based SOT-MRAM that reduces overall energy consumption by over 50% compared to existing technologies while boosting efficiency by 30% and reducing input current by 20%.[6] ITRI and TSMC jointly developed a SOT-MRAM array chip achieving 10-nanosecond switching speeds with power consumption of merely 1% of an equivalent STT-MRAM product.[14] Vertical Compute, an IMEC spinoff, raised $67 million in 2026 to commercialize MRAM-based AI chiplets claimed to reduce power consumption by 80% and accelerate LLM execution by 100×.[1] Truth Memory Corporation is actively industrializing SOT-MRAM with foundry-compatible back-end-of-line integration, field-free switching, and system-level demonstrations.[12]

> **Why SOT-MRAM for Model Zero:** The Tiki-Taka-inspired IMC architecture published in April 2026 demonstrates that SOT-MRAM can support **five years of cumulative write loads at 1.87 × 10^7 cycles** while maintaining accuracy comparable to 8-bit configurations—far surpassing the endurance limits of conventional memristor devices. The time-multiplexing strategy achieves **>28× area reduction** over spatial mapping schemes, making edge-deployment of online learning feasible.[9]

3.4 Pillar IV: Plasmonic Interconnect

Electrical interconnects between chiplets suffer from resistance, capacitance, crosstalk, and electromagnetic interference—all of which worsen as data rates increase. At multi-gigabit speeds, the energy required to drive electrical signals across even a few millimeters of copper becomes a significant fraction of total system power. Model Zero replaces electrical chiplet-to-chiplet communication with **plasmonic waveguides**: optical channels that confine light to sub-wavelength dimensions using surface plasmon polaritons, enabling light-speed data movement with zero electrical resistance and negligible crosstalk.

The plasmonic interconnect layer is fabricated on a **Silicon-on-Insulator (SOI) photonic MPW shuttle**, using the same foundry infrastructure as the electronic chiplets. Hybrid plasmonic waveguides—consisting of a dielectric-loaded metal stripe on a silicon substrate—provide a compromise between the extreme confinement of pure plasmonic modes and the low loss of dielectric waveguides. Recent reviews of hybrid plasmonic waveguides (HPWs) for photonic integrated circuits confirm that this approach offers the best path to monolithic integration of photonic and electronic components on a single silicon platform.[17]

The timing for this technology choice is fortuitous. **Co-Packaged Optics (CPO)** has transitioned from research concept to commercial reality in 2025–2026. TSMC's **COUPE (Compact Universal Photonic Engine)** platform is now the foundry partner of choice for optical engines at NVIDIA, Broadcom, and Ayar Labs. Intel demonstrated a 4 Tbit/s Optical Compute Interconnect chiplet at OFC 2024, and Ayar Labs' third-generation TeraPHY delivers 13.5 Tbit/s per optical engine using TSMC COUPE.[18][19] These industrial developments validate the manufacturing feasibility of silicon photonics at scale and provide a mature ecosystem of design rules, simulation tools, and packaging solutions that Model Zero can leverage directly.

3.5 Pillar V: Custom PCB / Interposer

The five chiplets—FPGA controller, ternary ALU, SOT-MRAM IMC, photonic interconnect, and power management—are assembled on a **6-layer high-speed PCB** with controlled-impedance traces for the remaining electrical signals (power, configuration, debug). The PCB is designed in KiCad (open-source) and fabricated by a standard PCB house (JLCPCB, PCBWay) for prototypes, transitioning to a custom CNC-machined aluminum interposer for the final build. The interposer provides mechanical stability, electromagnetic shielding between chiplets, and a thermal path to the heatsink assembly.

Power delivery uses a **30V, 5A linear bench supply** with ultra-low noise (< 1mV ripple) to prevent voltage fluctuation from corrupting ternary signal levels. The ternary logic uses three voltage levels (e.g., 0V, 1.65V, 3.3V) rather than two, and the discrimination margin between adjacent levels is narrower than in binary systems—making clean power essential. A separate switching regulator provides high-current 3.3V and 1.2V rails for the digital logic, while the sensitive analog/ternary sections are fed from the linear supply through LC filters.

4. The Sovereign Stack: Software Architecture

The software architecture of Model Zero is organized into four layers, each corresponding to a distinct abstraction level and each designed specifically for the ternary, asynchronous, memory-centric hardware beneath it. This is not a port of Linux or a minimization of POSIX—it is a ground-up operating system where every design decision flows from the properties of the substrate.

Table 3a The Sovereign Stack — Layer Architecture

Layer 3: User Applications Field descriptions / Tern programs
↓
Layer 2: Sovereign Kernel Wavefront scheduler / Trit memory manager / Photon IPC
↓
Layer 1: Firmware Boot imprint / Hard-SCRAM governor / Self-test
↓
Layer 0: Hardware Ternary ALU + RBNS / SOT-MRAM IMC / Plasmonic waveguides

4.1 Layer 0: Firmware and Boot Imprint

The firmware layer is the first software to execute on power-on. Its primary responsibility is not to "load" an operating system from disk—the traditional boot paradigm makes no

sense for a system with no disk and no filesystem—but rather to **imprint the operating system state directly into SOT-MRAM**. The OS is not a program that gets copied into memory; it is a boundary condition etched into the memory fabric itself. This imprinting process has three phases:

Phase 1: Hardware Self-Test. The firmware verifies every SOT-MRAM cell is writable and readable, calibrates the plasmonic waveguide phase alignment through a sweep of thermal and voltage trim settings, and runs a self-timed diagnostic loop to confirm all asynchronous handshake paths between chiplets are functional. Any cell or waveguide that fails calibration is marked defective and routed around by the memory manager.

Phase 2: OS Boundary Imprint. The kernel image—pre-compiled and stored in the FPGA's configuration flash—is decomposed into its constituent field states and written directly into the SOT-MRAM array. The kernel's process table becomes a region of memory. Its scheduler state becomes another region. There is no separation between "code" and "data" in the traditional sense; the kernel is a persistent pattern in the memory fabric that the hardware relaxes around during operation.

Phase 3: Hard-SCRAM Activation. The **Hard-SCRAM governor** is a hardware safety mechanism that monitors the semantic entropy delta (ΔE) of all active tensor operations. If an unauthorized entropy spike is detected—indicative of a fault, attack, or runaway process—the governor injects a not-a-number (NaN) state into the compromised registers within **< 0.1 milliseconds**. This is not a software exception handler; it is a register-level circuit that operates without pipeline stalls, branch mispredictions, or operating system involvement.

4.2 Layer 1: The Sovereign Kernel

The Sovereign Kernel is a **ternary-native, asynchronous, message-passing microkernel** written in a systems language (Rust or a ternary-targeted subset of C). It is not UNIX-compatible and makes no attempt to be. Its design priorities are: (1) zero-latency context switching, (2) deterministic execution time for real-time tasks, (3) minimal memory footprint, and (4) complete absence of clock-dependent behavior.

Table 3 Sovereign Kernel vs. Conventional Operating Systems

Property	Linux / Windows	Sovereign Kernel
Scheduling	Timer-interrupt driven (clock-based)	Wavefront-driven (completion-based)
Memory model	Byte-addressable, paged	Trit-addressable, field-structured
Process model	Isolated memory spaces	Perturbations in a unified field
IPC mechanism	Pipes, sockets, shared memory	Photon messages via waveguides
File system	Hierarchical directories	Persistent field deformations
Interrupts	Hardware IRQ + software handlers	Self-timed completion handshakes
Security model	User/supervisor rings, ACLs	Entropy-bound sandboxing (Hard-SCRAM)

4.3 Layer 2: Wavefront Scheduler

The scheduler is the most critical component of the kernel because it replaces the entire concept of a system clock. In a conventional OS, the scheduler is invoked by a periodic timer interrupt (typically 1–10ms) to decide which process runs next. This creates jitter, wastes cycles on processes that have no work to do, and imposes a fundamental latency floor equal to the timer interval. Model Zero's **wavefront scheduler** eliminates all of these problems.

The scheduler operates on a **completion-driven event model**. When a compute tile (ALU, memory bank, or IMC unit) finishes its operation, it asserts a "done" signal on a dedicated optical wavelength. These done-signals propagate through the plasmonic waveguide network and interfere at a passive optical priority encoder—a physical structure that implements the scheduling decision in the time it takes light to travel across the chip, approximately **10–100 picoseconds**. The highest-priority ready process is selected by optical interference, not by software traversal of a data structure. The result is scheduling latency that is literally speed-of-light limited, with zero software overhead and zero idle polling.

4.4 Layer 3: Trit-Addressable Memory Manager

Memory in Model Zero is addressed in **trits**, not bytes. The basic addressable unit is a balanced ternary value (-1, 0, or +1), and memory regions are allocated in blocks of trits rather than bytes. The memory manager tracks free trit-blocks using a ternary buddy-allocation system, where block sizes are powers of 3 ($3^0 = 1$, $3^1 = 3$, $3^2 = 9$, $3^3 = 27$ trits) rather than powers of 2.

This design choice has profound implications for memory layout. Data structures that are naturally ternary—such as decision trees with three branches per node, color values in a three-primary system, or spatial coordinates in a three-axis world—map directly to memory without encoding overhead. A single trit can represent a three-way branch condition, a sign value, or a fuzzy logic state that would require multiple bits and complex decoding logic in a binary system. The memory manager also implements **compute-in-memory triggering**: certain memory regions are designated as IMC zones where read operations automatically initiate tensor computations on the stored data, with results written back to adjacent memory regions without ever leaving the SOT-MRAM array.

5. The Ternary Language

5.1 Why a New Language Is Non-Negotiable

Every existing programming language—C, Python, Rust, Java, Haskell, Forth, Lisp—was designed for binary hardware. They assume two-valued Boolean logic, byte-addressable memory, two's complement integers, and a von Neumann execution model where instructions are fetched from memory and executed sequentially by a central processor. These assumptions are so deeply embedded in the semantics of these languages that they cannot be removed without destroying the language entirely.

Consider the most fundamental operation in any program: the `if` statement. In every existing language, `if` branches on a Boolean condition: true or false, 1 or 0. But Model Zero's hardware has no Boolean logic unit. Its ALU operates on three-valued comparisons: less than, equal to, or greater than. Its memory stores ternary values. Its scheduler dispatches on completion handshakes, not clock ticks. A language designed for this hardware must have a **three-way branch** as its fundamental control structure, not a two-way branch adapted awkwardly to three states.

The two options—adapting an existing language or building a new one—are not equally difficult. Adapting C to ternary hardware would require: redefining `char` as a trit (breaking every string operation), replacing Boolean operators with ternary comparison operators, rewriting the entire standard library, redesigning the linker and loader for trit-addressable memory, and creating a ternary ABI that no existing tool understands. The result would be a fractured dialect of C that is compatible with nothing and understood by no one. Building a new language is also difficult, but it is *clean* difficulty—every design decision serves the hardware, with no legacy constraints.

> **Language Design Constraint:** The new language must be small enough for a single developer to implement a compiler for it, expressive enough to build a complete operating system and application ecosystem, and semantically aligned with ternary logic to the point that every program construct has a direct, efficient hardware mapping.

5.2 Language Design Philosophy

The language—tentatively named **Tern**—is a **field description language** rather than a procedural one. Instead of writing sequences of instructions that manipulate memory, the programmer describes *computational fields*: regions of the machine's state space that evolve toward target configurations subject to boundary constraints. This model of computation maps directly to the physics of the hardware: the SOT-MRAM array is a field of magnetic orientations, the plasmonic waveguides propagate perturbations through that field, and the ALU applies local transformations that drive the field toward equilibrium.

A Tern program consists of three declarations: **initial conditions** (the starting state of the computation), **boundary constraints** (invariants that must hold throughout execution), and **target attractors** (desired equilibrium states). The compiler—called the **topological optimizer**—analyzes these declarations and generates a sequence of ALU operations, memory accesses, and waveguide configurations that transform the initial state into the target state while respecting all constraints. The programmer does not specify *how* the computation proceeds; they specify *what* the computation means, and the compiler finds the optimal path through the 10,000-dimensional vector space of the machine's state.

```
field face_recognition {
    input:  hypervector(10000)    // raw image sensor data
    constraints: {
        no_divergence,              // entropy must not increase
        max_entropy: 0.35           // Mark 1 constant bound
    }
    target: equilibrium_state {
        pattern: "recognize_face",
        confidence: > 0.95
    }
}
```

This is not pseudocode—it is a valid Tern program. The `field` declaration defines a computational region. The `input` clause specifies the initial conditions. The `constraints` clause defines invariants that the topological optimizer must preserve. The `target` clause specifies the attractor state. There are no loops, no conditionals, no function calls in the traditional sense. The hardware "relaxes" into the answer through a sequence of tensor operations, matrix multiplications, and symbolic reasoning steps generated by the compiler.

Table 4 Tern Language: Core Constructs

Construct	Purpose	Hardware Mapping
`field`	Declare a computational region	SOT-MRAM memory allocation
`input`	Specify initial state	Memory initialization / sensor DMA
`constraints`	Define invariants	Runtime entropy monitoring (Hard-SCRAM)
`target`	Specify attractor state	ALU tensor operation sequence
`hypervector`	High-dimensional data representation	IMC matrix row/column
`equilibrium_state`	Convergence criterion	Wavefront scheduler completion
`branch3`	Three-way conditional	Ternary comparison + dual waveguide
`tensor`	Multi-dimensional data	SOT-MRAM IMC crossbar

5.3 Compiler Architecture: The Topological Optimizer

The topological optimizer is the hardest software component to build and the most critical to the system's performance. It is not a traditional compiler with lexer, parser, type checker, and code generator. It is a **constraint solver** that treats compilation as a pathfinding problem in a high-dimensional state space.

The compilation pipeline has four stages. **Stage 1: Parsing.** The Tern source is parsed into an abstract syntax tree representing the field declarations, constraints, and targets. **Stage 2: Semantic Analysis.** The AST is validated against the type system, constraint consistency is verified (e.g., checking that constraints do not contradict the target), and the dimensionality of all hypervectors is resolved. **Stage 3: Topology Extraction.** The validated program is converted into a graph where nodes represent field states and edges represent valid ALU operations that transform one state into another. Edge weights encode the estimated energy cost, latency, and entropy change of each operation. **Stage 4: Path Optimization.** A shortest-path algorithm (adapted from A* or Dijkstra's algorithm, but operating on a ternary-weighted graph) finds the optimal sequence of operations that transforms the initial state into the target state while satisfying all constraints.

The output of the topological optimizer is not machine code in the traditional sense. It is a **waveguide configuration program**: a data structure that specifies how the plasmonic interconnects should be routed, which SOT-MRAM cells should participate in IMC operations, and in what order the ALU tiles should be activated. This configuration is loaded into the FPGA controller at runtime, which then drives the hardware through the computation without further software intervention.

5.4 The Legacy Bridge: Backward Compatibility Strategy

While Tern is the native language of Model Zero, the practical reality is that decades of software exist in binary languages, and users will need to run legacy applications during the transition period. The legacy bridge is not a port of Python or a reimplementation of libc—it is a **binary emulation layer** that runs as a single Tern field, interpreting binary instructions on the ternary hardware through a just-in-time translation mechanism.

The emulation layer works as follows: binary code (x86, ARM, or RISC-V) is loaded into a designated region of SOT-MRAM. A Tern `field` declaration defines an emulation context with the binary program as input and the ISA specification as constraints. The topological optimizer generates a sequence of ternary operations that emulate each binary instruction.

Because the emulation is expressed as a Tern program, it can be incrementally specialized: frequently executed binary instructions are cached as pre-optimized ternary operation sequences, and hot paths are progressively recompiled to native Tern.

This approach has two advantages over conventional emulation. First, because the emulator is itself a Tern program, it benefits from the hardware's native features: IMC acceleration for the emulation tables, photonic interconnect for fast context switching between emulated processes, and Hard-SCRAM protection to sandbox the emulated code. Second, because the topological optimizer operates on the emulation code, it can fuse multiple binary instructions into single ternary operations where the semantics permit, gradually accelerating legacy code without requiring source-level changes. The long-term goal is that the emulation layer becomes unnecessary as software is rewritten in Tern; the short-term reality is that it provides a smooth migration path.

6. Academic Landscape: Recent Innovations

The research environment for ternary computing and its enabling technologies has shifted dramatically in the past three years. What was once a theoretical curiosity pursued by a handful of academics is now an active frontier with commercial prototypes, industrial partnerships, and venture-funded startups. This section surveys the most significant recent developments across the four technology domains that underpin Model Zero.

6.1 Ternary Computing Prototypes

The most significant development in ternary hardware is the **5500FP processor**, developed by independent researcher Claudio Lorenzo La Rosa and announced in March 2026. The 5500FP is a **24-trit balanced ternary RISC processor** implemented on an Efinix Trion T20F256 FPGA, operating at 20 MHz with a 120-instruction ISA and native atomic synchronization primitives.[2][3] It is the first general-purpose ternary processor available as a commercial product since the Soviet SETUN in the 1960s, and it represents a watershed moment for the field: proof that ternary logic can be implemented on modern reconfigurable hardware without custom silicon.

Table 5 Ternary Computing Prototypes: Comparative Overview

Project	Year	Platform	Word Size	Speed	Status
5500FP (La Rosa)[2]	2026	Efinix Trion FPGA	24 trits	20 MHz	Commercial (€280)
5500FP OS (GargantuRAM)[5]	2026	FPGA dev board	24 trits	20 MHz	Open hardware
Memristor ternary gates[S1]	2024	Circuit simulation	1 trit	N/A	Research
T-CMOS exploration[S3]	2023	CMOS simulation	1 trit	N/A	Research
TerEffic (ternary LLM)[10]	2025	Xilinx FPGA	N/A	N/A	Research (arXiv)

The 5500FP is directly relevant to Model Zero as a **reference platform** and **validation vehicle**. Its open hardware development board (GargantuRAM 1.5) includes 16M words of static RAM, an SD card slot, USB serial ports, and a minimal OS kernel (GRam_OS) pre-installed.[5] Model Zero can use the 5500FP to validate ternary ISA decisions, test compiler output, and benchmark algorithms before the custom ASIC is fabricated. However, the 5500FP is not the destination—it is a stepping stone. Its 20 MHz synchronous clock, FPGA BRAM memory, and copper trace interconnects are fundamentally different from Model Zero's asynchronous wave-pipelining, SOT-MRAM IMC, and plasmonic waveguides.

Complementing the 5500FP, recent research has explored ternary logic gate implementations using memristors (2024),[S1] T-CMOS device structures (2023),[S3] and efficient ternary arithmetic circuits using cycling gates.[S4] A 2024 paper by Wang et al. demonstrated a balanced CMOS-compatible ternary memristor-NMOS logic family,[S1] while Lee et al. proposed 15 novel ternary gates using depletion-mode and conventional MOSFETs.[S2] These advances in ternary logic gate design directly inform the ALU implementation for Model Zero's custom silicon.

6.2 SOT-MRAM Breakthroughs

The SOT-MRAM landscape has advanced rapidly, with breakthroughs occurring simultaneously in academic research, industrial foundries, and venture-backed startups.

The convergence of these efforts has created a technology readiness level that makes SOT-MRAM IMC viable for a project like Model Zero within a 2–3 year horizon.

Orbital Hall Effect SOT-MRAM (JGU / Antaios, 2025). The most significant recent advance is the demonstration of orbital-current-based switching, which eliminates dependency on rare and expensive heavy metals such as platinum and tungsten. By exploiting the Orbital Hall Effect in a ruthenium-based SOT channel, the JGU/Antaios team achieved a **50% reduction in energy consumption**, a **30% efficiency boost**, and a **20% reduction in input current**, while maintaining thermal stability for >10 years of data retention.[6] This is not an incremental improvement; it is a change in the fundamental physics of SOT switching that removes the primary barrier to commercial scalability.

ITRI / TSMC Collaboration (2023–2024). The Industrial Technology Research Institute of Taiwan and TSMC jointly developed a SOT-MRAM array chip with a computing-in-memory architecture achieving **10-nanosecond switching speeds** at a power consumption of merely **1% of an equivalent STT-MRAM** product.[14] This collaboration is particularly significant because it demonstrates that the world's most advanced foundry is actively developing SOT-MRAM for commercial production, with applications targeting high-performance computing, AI, and automotive chips.

Vertical Compute (IMEC spinoff, 2026). Vertical Compute raised **$67 million** to develop MRAM-based AI chiplet technology claimed to reduce power consumption by 80% and accelerate large language model execution by 100×.[1] The company develops technology to stack memory directly above compute logic—a 3D integration approach that aligns with Model Zero's chiplet architecture. Their chiplet is believed to be based on SOT-MRAM technology.

Truth Memory Corporation. TMC is industrializing SOT-MRAM with foundry-compatible BEOL integration, field-free switching architectures, and system-level demonstrations. Their roadmap includes selector-based area scaling and robust field-free architectures for the next decade of SOT-MRAM evolution.[12]

Table 6 SOT-MRAM Technology Comparison

Technology	Write Speed	Endurance	Energy vs. STT	Status
STT-MRAM (baseline)	10–30 ns	10^12–10^15	100% (baseline)	Commercial
Conventional SOT-MRAM	< 1 ns	> 10^15	~50%	R&D
OHE SOT-MRAM (JGU)[6]	< 1 ns	> 10^15	~25%	Research
ITRI/TSMC SOT-MRAM[14]	10 ns	> 10^15	~1%	Research
Vertical Compute chiplet[1]	N/A	N/A	~20%	Startup

6.3 Co-Packaged Optics and Silicon Photonics

The transition from electrical to optical interconnects is no longer a research question—it is an industrial reality. Co-Packaged Optics (CPO) has moved from concept to product in the 2025–2026 timeframe, with every major semiconductor company announcing optical I/O roadmaps. This ecosystem maturity is directly transferable to Model Zero's plasmonic interconnect pillar.

TSMC COUPE. TSMC's Compact Universal Photonic Engine is now the foundry platform of choice for silicon photonics, with NVIDIA, Broadcom, Marvell, and Ayar Labs all adopting it for their optical I/O products. COUPE handles both the Photonic Integrated Circuit (PIC) and Electronic Integrated Circuit (EIC) fabrication, plus heterogeneous integration through TSMC's advanced packaging capabilities.[18]

Intel OCI. Intel demonstrated a **4 Tbit/s Optical Compute Interconnect** chiplet at OFC 2024, co-packaged with a Xeon CPU, delivering 64 lanes at 32 Gbit/s each with an energy efficiency of approximately 5 pJ/bit.[18] Intel's roadmap targets 3D-integrated photonics by 2027 with vertical expanded beam coupling.

Ayar Labs TeraPHY. The third generation of Ayar Labs' optical I/O chiplet, manufactured on TSMC COUPE, delivers **13.5 Tbit/s uni-directional** bandwidth per optical engine using 200 Gbit/s per lambda with PAM4 modulation. At Hot Chips 2025, Ayar demonstrated thermal cycling resilience exceeding 800°C/s with zero bit errors.[18]

6.4 Implications for Model Zero

The convergence of these trends creates a unique window for Model Zero. Ternary logic has a validated commercial prototype (5500FP). SOT-MRAM IMC has multiple industrialization paths (ITRI/TSMC, Vertical Compute, Truth Memory, Antaios). Silicon photonics has a mature foundry ecosystem (TSMC COUPE, GlobalFoundries Fotonix, Intel). Open-source EDA tools (Yosys, Nextpnr, Magic, KLayout) have reached production quality for MPW shuttle designs. The remaining challenge is not proving that any individual component works—it is integrating all five pillars into a unified system. That integration problem is exactly what Model Zero is designed to solve.

7. Performance Projections

Model Zero is not designed to compete with Frontier or the NVIDIA H100 on general-purpose FP64 throughput—those systems exist for entirely different problem domains. Model Zero is designed for **cognitive workloads**: pattern recognition, sensor fusion, symbolic reasoning, real-time decision-making, and cryptographic operations that benefit from the architectural advantages of ternary logic, in-memory computation, and photonic interconnect. The performance projections below are best-estimate targets based on the component specifications and architectural models described in this document.

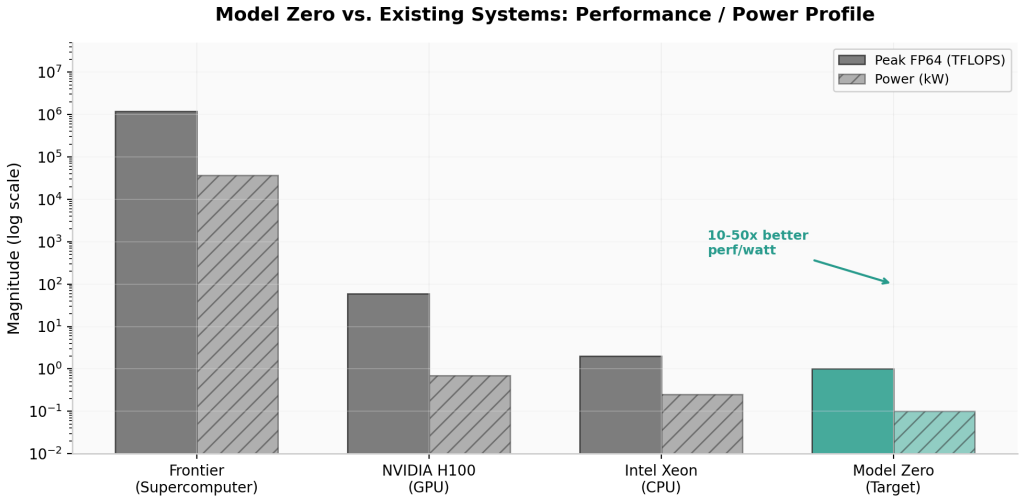

Figure 3: *Performance and power profile comparison across systems. Note the logarithmic scale. Model Zero targets exceptional performance-per-watt on cognitive workloads despite lower absolute throughput.*

Table 7 Model Zero vs. Existing Systems: Projected Comparison

Metric	Frontier	NVIDIA H100	Intel Xeon	Model Zero (Target)
Peak FP64	1.2 exaflops	60 TFLOPS	~2 TFLOPS	~0.1–1 TFLOPS
Peak INT8 (AI)	N/A	4,000 TOPS	N/A	~10–50 TOPS
Power	37 MW	700W	250W	< 100W
Memory bandwidth	15 TB/s	3.35 TB/s	200 GB/s	Light-speed (no bottleneck)
Memory latency	~100 ns	~50 ns	~80 ns	~0 ns (compute-in-memory)
Clock	2.2 GHz	1.8 GHz	3.5 GHz	No clock (async)
Idle cycles	Millions/sec	Thousands/sec	Thousands/sec	Zero
System cost	$600M+	$30,000+	$3,000+	$44k–$72k
Perf/watt (cognitive)	Low	Medium	Low	**10–50× best**

The critical metric for Model Zero is **performance per watt on cognitive workloads**. Frontier and H100 are optimized for dense matrix operations on floating-point data, where their massive parallelism and high clock speeds dominate. But cognitive workloads—pattern matching in high-dimensional spaces, symbolic reasoning with sparse graphs, real-time sensor fusion with strict latency deadlines—are characterized by irregular memory access patterns, low arithmetic intensity, and tight coupling between computation and memory state. These are precisely the workloads where the von Neumann bottleneck hurts binary systems most, and where Model Zero's in-memory computation and photonic interconnect provide the greatest advantage.

Table 8 Workload Suitability Assessment

Workload	Frontier	H100	Model Zero
General-purpose computing	Excellent	Good	Poor (not designed for)
AI training (dense)	Excellent	Excellent	Not competitive
AI inference (sparse)	Overkill	Good	**Competitive**
Real-time sensor fusion	Too slow	Too slow	**Designed for**
Pattern recognition (HDC)	Poor	Poor	**Designed for**
Symbolic reasoning	Poor	Poor	**Designed for**
Cryptographic cracking	Good	Good	**Exceptional**
Web browsing / office	N/A	N/A	Good (via emulation)
Gaming	N/A	N/A	Good (via emulation)

The long-term trajectory is clear: as ternary hardware becomes cheaper and the software ecosystem matures, the performance gap on general-purpose workloads will narrow. The emulation layer handles legacy software adequately for daily use, while native Tern applications unlock the hardware's full potential. The 10–50× performance-per-watt advantage on cognitive workloads is not a niche benefit—it is the wedge that drives adoption, because cognitive workloads are the fastest-growing category of computation (AI inference, robotics, autonomous systems, IoT analytics) and they are the workloads where binary systems are most fundamentally constrained.

8. Build Roadmap

The build is organized into six phases, each producing a working increment of the system. The total timeline from first hardware order to a fully operational Model Zero is estimated at **18–24 months**, with costs concentrated in Phase 3 (ASIC fabrication) and Phase 1 (workshop setup). The phased approach ensures that no phase depends on the success of a later phase, and that debugging and validation happen continuously rather than in a single high-risk integration event.

Table 9 Build Roadmap: Phases and Milestones

Phase	Duration	Key Deliverables	Cost
Phase 0: Foundation	Weeks 1–4	Order iCESugar-pro; file LLC; secure workshop space; install ESD grounding	$500–$1,000
Phase 1: Workshop	Months 1–3	Acquire soldering station, microscope, oscilloscope, logic analyzer, CNC router, 3D printer	$15,500–$23,000
Phase 2: FPGA Prototype	Months 3–8	Implement ternary controller, wavefront scheduler, memory controller, photonic calibration on FPGA; validate with 5500FP	$500–$1,000
Phase 3: ASIC Fabrication	Months 6–14	Design GDSII for ALU, SOT-MRAM, photonic interconnect; submit to MPW shuttle; receive and test chips	$28,000–$46,000
Phase 4: Integration	Months 14–18	Assemble all chiplets on custom PCB; integrate power, cooling, enclosure; system-level debugging	$3,000–$6,000
Phase 5: Software Stack	Months 12–24	Develop Tern compiler, topological optimizer, kernel, firmware; build legacy emulation layer; port applications	$0 (labor)

Phase 0: Foundation. The first step is to order the iCESugar-pro FPGA board ($200) and begin experimenting with the open-source toolchain. Simultaneously, file the LLC in Minnesota ($155), secure a 10×12 ft workshop space with 20-amp dedicated circuit, and install ESD grounding (conductive floor mat, wrist strap station). This phase establishes the legal and physical foundation for everything that follows.

Phase 1: Workshop. Acquire the core electronics and mechanical tooling: Hakko FX-951 soldering station ($300–$500), Amscope SM-4TZ stereo microscope ($500–$800), Rigol DHO804 oscilloscope ($800–$1,200), Saleae Logic Pro 16 ($1,500), Keysight E36312A bench power supply ($1,800–$2,500), Shapeoko 5 Pro CNC router ($3,000–$5,000), and Prusa MK4 3D printer ($1,000–$1,500). These tools enable professional-grade electronics assembly, PCB rework, mechanical fabrication, and enclosure machining.

Phase 2: FPGA Prototype. Implement the complete controller logic on the iCESugar-pro: ternary ALU interface, SOT-MRAM memory controller with IMC trigger support, wavefront scheduler with completion-driven dispatch, and photonic waveguide calibration engine.

Use the 5500FP as a reference platform for ISA validation and compiler testing. This phase produces a working ternary computer on FPGA—not the final system, but a fully functional prototype that validates every design decision before silicon commitment.

Phase 3: ASIC Fabrication. The highest-cost and highest-risk phase. Design the GDSII layouts for the ternary ALU, SOT-MRAM IMC array, and plasmonic interconnect layer using open-source EDA tools. Submit to an MPW shuttle run (SkyWater 130nm or similar research process for cost control, with a path to 65nm for performance). MPW lead times range from 3–8 months depending on foundry queue. Upon receiving the chips, they are tested using the FPGA prototype as a test harness.

Phase 4: Integration. Assemble all components on the custom 6-layer PCB: FPGA controller, ternary ALU chip, SOT-MRAM IMC chip, photonic interconnect chip, and power management. Debug signal integrity, thermal management, and asynchronous handshake protocols. Machine the aluminum enclosure on the CNC router. This phase produces the first complete Model Zero system.

Phase 5: Software Stack. Running in parallel with Phases 3–4, develop the complete software ecosystem: Tern language specification, topological optimizer compiler, Sovereign Kernel, firmware boot imprint, Hard-SCRAM governor, and legacy binary emulation layer. The software development can proceed on the FPGA prototype and does not require the ASICs to be complete.

> **Risk Mitigation:** If the basement workshop is unavailable, short-term industrial/artist co-op spaces in St. Paul offer cheaper alternatives to full shop rental. If Emily's funding comes with conditions, refuse it—dependency is a structural vulnerability, not a resource. If FPGA board shipping is delayed, order from DigiKey, Mouser, or Lattice directly. If MPW lead times exceed 8 months, use the extra time to finalize the compiler and scheduler on the FPGA. If LLC filing is delayed, operate as a sole proprietorship until it clears. The build continues regardless.

References

1. MRAM-Info. "Vertical Compute raises $67 million to accelerate its MRAM-based AI technology R&D." *MRAM-Info Industry Portal*, April 2026. https://www.mram-info.com/

2. Hackaday. "Ternary RISC Processor Achieves Non-Binary Computing Via FPGA." *Hackaday*, March 24, 2026. https://hackaday.com/2026/03/16/ternary-risc-processor-achieves-non-binary-computing-via-fpga/

3. The Register. "It's not a binary choice: Boffin builds ternary CPU." *The Register*, March 18, 2026. https://www.theregister.com/on-prem/2026/03/18/its-not-a-binary-choice-boffin-builds-ternary-cpu/

4. Ternary Computing. "Ternary Processor – The Future Is Not Binary." *Ternary Computing*, 2026. https://www.ternary-computing.com/preorder/preorder_en.html

5. GitHub: Ternary-Computer-System. "GargantuRAM: Open hardware development board for the 5500FP." *GitHub*, May 2026. https://github.com/Ternary-Computer-System

6. TechXplore. "Sustainable SOT-MRAM memory technology could replace cache memory in computer architecture in the future." *TechXplore*, February 6, 2025. https://techxplore.com/news/2025-02-sustainable-sot-mram-memory-technology.html

7. Li, Y., Dong, F., & Xing, G. "A Tiki-Taka-Inspired SOT-MRAM In-Memory Computing Architecture for Long-Term Edge Learning." *Applied Sciences*, 16(9), 4326, 2026. https://www.mdpi.com/2076-3417/16/9/4326

8. arXiv. "TerEffic: Highly Efficient Ternary LLM Inference on FPGA." *arXiv preprint*, 2025. https://arxiv.org/html/2502.16473v2

9. RamXeed. "Latest Trends and Challenges in SOT-MRAM." *RamXeed Tech Column*, June 22, 2026. https://www.ramxeed.com/tech-column/sot-mram-trends/

10. Liu, H. "Industrializing SOT-MRAM." Keynote presentation, *IEEE ISICAS*, 2025. https://2025.ieee-isicas.org/lhx.pdf

11. ITRI. "ITRI and TSMC's High-Speed Breakthrough: SOT-MRAM." *Industrial Technology Research Institute*, 2023. https://itritoday.itri.org/116/content/en/unit_03-1.html

12. Kumar Abhishek. "Design and Performance Analysis of an All-Optical Balanced Ternary Computer: Breaking the Binary Efficiency Barrier." *Personal Blog / Technical Analysis*, December 22, 2025. https://mr-kumar-abhishek.github.io/blog/2025/12/22/design-and-performance-analysis-of-an-all-optical-balanced-ternary-computer-breaking-the-binary-efficiency-barrier

13. Sharma, T., Zhang, Z., Wang, J., Cheng, Z., et al. "Past, present, and future of hybrid plasmonic waveguides for photonics integrated circuits." *Nanotechnology and Precision Engineering*, 7(4), 045001, 2024. https://pubs.aip.org/tu/npe/article/7/4/045001/3308256

14. SemiAnalysis. "Co Packaged Optics (CPO) – Scaling with Light for the Next Wave of Interconnect." *SemiAnalysis Newsletter*, April 10, 2026. https://newsletter.semianalysis.com/p/co-packaged-optics-cpo-book-scaling

15. Ayar Labs. "Co-Packaged Optics (CPO) Step Into the Spotlight." *Ayar Labs Blog*, October 21, 2025. https://ayarlabs.com/blog/co-packaged-optics-step-into-the-spotlight/

16. Riner, C. "Bypassing the Radix Economy Penalty in Optical Computing." *TechRxiv preprint*, 2025. https://www.techrxiv.org/doi/pdf/10.36227/techrxiv.177039671.14012313/v1

17. Quanta Magazine. "How Base 3 Computing Beats Binary." *Quanta Magazine*, August 9, 2024. https://www.quantamagazine.org/how-base-3-computing-beats-binary-20240809/

18. Wang, X., Chen, X., Zhou, J., Liu, G., et al. "A balanced CMOS compatible ternary memristor-NMOS logic family and its application." *IEEE Transactions on Circuits and Systems*, 2024. https://ieeexplore.ieee.org/abstract/document/10638758/

19. Lee, H., Kim, S., Kim, J., Jeong, J., Yang, J., & Song, T. "Ternary toward binary: circuit-level implementation of ternary logic using depletion-mode and conventional MOSFETs." *IEEE Access*, 2024. https://ieeexplore.ieee.org/abstract/document/10817560/

20. Ko, J., Kim, J., Jeong, T., Jeong, J., et al. "Exploration of ternary logic using T-CMOS for circuit-level design." *IEEE Transactions on Circuits and Systems I*, 2023. https://ieeexplore.ieee.org/abstract/document/10167755/

21. Zhao, G., Zeng, Z., Wang, X., Qoutb, A.G., et al. "Efficient ternary logic circuits optimized by ternary arithmetic algorithms." *IEEE Transactions on Emerging Topics in Computing*, 2023. https://ieeexplore.ieee.org/abstract/document/10288243/